第 1 章　Python で天体の運動を計算する

銀河相互作用の計算を始める前に、重力を受ける 1 個の粒子を動かしてみましょう。この章では、原点に固定した中心質量の周りを回る粒子を考えます。運動方程式を Python で計算できる形に直し、重力計算の標準法である leapfrog 法（KDK）を実装します。一般の常微分方程式に使える 4 次 Runge–Kutta 法（RK4）も学び、同じ軌道で両者を比較します。最後に、解析解、保存量、時間刻み、力評価回数から、計算結果と手法の使い分けを考えます。

この章で学ぶこと

- 位置・速度・加速度を用いて、運動方程式を 1 階の連立常微分方程式として書けます。
- leapfrog の KDK 更新と RK4 の 4 段階を説明し、両方を Python で実装できます。
- 軌道を図とアニメーションで表示し、同じ時刻の解析解と比較できます。
- エネルギー・角運動量・時間刻みを使って、計算結果を確認できます。
- 精度だけでなく力評価回数も比べ、重力計算と速度依存力で手法を使い分けられます。

1.1 数値シミュレーションの目的を考える

物体に働く力が分かれば、Newton の運動方程式を書くことができます。しかし、方程式を書いただけでは、しばらく後に物体がどこにいるかは分かりません。初めの位置と速度を与え、運動方程式を解く必要があります。このように、ある時刻の状態から後の状態を求める問題を**初期値問題**と呼びます。

ゼミでは、中心力のもとでの運動や保存量を学んできました。本章では、その知識を使って「粒子の位置と速度を、少し後の時刻へ更新する」という計算を作ります。これを繰り返すと、粒子の軌道が得られます。ここでいう**時間積分**とは、速度を積分して位置を、加速度を積分して速度を求め、状態を時間発展させることです。

最初の問題には、解析的に答えが分かる円軌道を選びます。答えが分かっている問題を数値的に解くのは、遠回りではありません。自分のコードが、正しい向きの力を計算し、適切な時刻に正しい場所へ粒子を運べるかを確認できるからです。これができてから、解析的に解くことが難しい銀河相互作用へ進みます。

本章の中心質量は、銀河全体の最終モデルではありません。第 3 章では質量に広がりを持たせた Plummer ポテンシャルへ、第 5 章では 2 つの移動する Plummer ポ

テンシャルへ進みます。**重力場の式は変わっても、加速度を計算して leapfrog で進める基本手順は再利用します。**

1.2 中心重力場で Newton の運動方程式を立てる

1.2.1 計算するモデルを決める

原点に質量 M の天体を固定し、その周りを質量 m の粒子が運動するとします。粒子が中心天体に及ぼす重力は無視します。これは、例えば $m \ll M$ で中心天体の動きが小さい場合を考える近似です。粒子を単に質量ゼロと置いて運動方程式を消すのではなく、粒子から中心への反作用を計算しない、という意味です。

座標系は原点に中心天体を置いた慣性系とし、運動は x–y 平面内に限定します。重力以外の力、中心天体の大きさ、他の天体の影響は含めません。この中心質量は時間とともに変化せず、原点も動きません。2 つの天体がともに動く問題は、第 2 章で扱います。

粒子の位置、速度、加速度を、それぞれ

$$\boldsymbol{r} = (x, y), \qquad \boldsymbol{v} = \frac{d\boldsymbol{r}}{dt} = (v_x, v_y), \qquad \boldsymbol{a} = \frac{d\boldsymbol{v}}{dt} = (a_x, a_y) \tag{1.1}$$

と書きます。太字はベクトル、細字の $r = |\boldsymbol{r}| = \sqrt{x^2 + y^2}$ は距離を表します。

中心質量が粒子を引く力は、中心からの距離の 2 乗に反比例し、原点を向きます。大きさ GMm/r^2 に原点方向の単位ベクトル $-\boldsymbol{r}/r$ を掛けると、Newton の運動方程式は

$$m\frac{d^2\boldsymbol{r}}{dt^2} = -\frac{GMm}{r^3}\boldsymbol{r} \tag{1.2}$$

となります。両辺の m を消去し、$\mu \equiv GM$ と置くと、加速度は

$$\boxed{\boldsymbol{a}(\boldsymbol{r}) = -\frac{\mu}{r^3}\boldsymbol{r}} \tag{1.3}$$

です。与えられた重力場中では、粒子の軌道は m に依存しません。

1.2.2 ベクトルの式を成分で読む

式(1.3)を、そのままコードに移せる形にします。x 方向と y 方向の加速度は

$$a_x = -\frac{\mu x}{(x^2 + y^2)^{3/2}}, \qquad a_y = -\frac{\mu y}{(x^2 + y^2)^{3/2}} \tag{1.4}$$

です。例えば $x > 0$, $y = 0$ なら $a_x < 0$, $a_y = 0$ です。力が正しく原点へ向いていることを、このような簡単な配置で確認できます。

ここで分母が r^3 であることと、重力の大きさが r^{-2} に比例することは矛盾しません。分子に長さ r のベクトルがあるため、式全体の大きさは μ/r^2 です。

よくある失敗：重力の向きと原点

位置ベクトルを原点から粒子へ向けて定義したので、加速度には負号が必要です。また、点質量の重力は $r = 0$ で定義できません。本章では原点へ衝突する軌道を扱わず、分母へ根拠なく小さな数を足して発散を隠すこともしません。力の式を変えれば、別のモデルになるためです。

1.3 運動方程式を 1 階の連立方程式にする

1.3.1 位置と速度を一つの状態として扱う

数値積分では、2 階微分を含む式(1.2)を直接扱う代わりに、位置と速度を独立した変数として持ちます。位置の変化率は速度、速度の変化率は加速度ですから、

$$\frac{d\boldsymbol{r}}{dt} = \boldsymbol{v}, \qquad \frac{d\boldsymbol{v}}{dt} = \boldsymbol{a}(\boldsymbol{r}) \tag{1.5}$$

と書けます。もとの運動方程式と内容は同じで、未知変数を増やして微分の階数を下げただけです。

位置と速度をまとめた**状態ベクトル**を $\boldsymbol{y} = (\boldsymbol{r}, \boldsymbol{v})$ とします。縦に成分を並べれば、この問題は

$$\boldsymbol{y} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}, \qquad \frac{d\boldsymbol{y}}{dt} = \boldsymbol{f}(t, \boldsymbol{y}) = \begin{pmatrix} v_x \\ v_y \\ -\mu x/(x^2+y^2)^{3/2} \\ -\mu y/(x^2+y^2)^{3/2} \end{pmatrix} \tag{1.6}$$

です。太字の $\boldsymbol{y}$ は状態ベクトル、細字の y は位置の y 成分です。また、状態ベクトルの位置成分と速度成分は同じ単位ではありません。右辺の上 2 成分は速度、下 2 成分は加速度を表します。

今回の重力場には、時刻 t が直接は現れません。それでも $\boldsymbol{f}(t, \boldsymbol{y})$ という書き方にしておくと、後の章で重力場が時間変化する場合にも同じ形式を使えます。速度に依存する加速度が加わっても、状態ベクトルの構成は変わりません。一方、重力だけを leapfrog で解くときは、位置と速度を分けて持ち、順序を決めて交互に更新します。

1.3.2 単位と円軌道の初期条件を決める

計算を始める前に、単位をそろえます。本章では距離を kpc、時間を Gyr、質量を $M_\odot$ とします。**コード内部の速度は** $\mathrm{kpc\,Gyr^{-1}}$ **です**。速度を天文学でよく使う $\mathrm{km\,s^{-1}}$ で表示するときだけ、

$$1\,\mathrm{km\,s^{-1}} \simeq 1.022712165\,\mathrm{kpc\,Gyr^{-1}} \tag{1.7}$$

を使います。重力定数も同じ単位系へ変換します。教材では近似値

$$G = 4.30091 \times 10^{-6}\,\mathrm{kpc}\,(\mathrm{km\,s^{-1}})^2\,M_\odot^{-1}$$

を固定して使い、解析解にも同じ値を使います。このため、以下の精度試験は物理定数の測定精度ではなく、数値解法の誤差を調べています。

半径 r_0 の円運動では、重力加速度の大きさと向心加速度が等しくなります。この条件から、円運動速度 v_c、角速度 Ω、周期 T は

$$\frac{v_c^2}{r_0} = \frac{\mu}{r_0^2}, \qquad v_c = \sqrt{\frac{\mu}{r_0}}, \qquad \Omega = \sqrt{\frac{\mu}{r_0^3}}, \qquad T = \frac{2\pi}{\Omega} \tag{1.8}$$

となります。この円軌道を、数値計算の基準にします。

本章では、$t = 0$ で粒子を正の x 軸上へ置き、正の y 方向へ速度を与えます。

$$\boldsymbol{r}(0) = (r_0, 0), \qquad \boldsymbol{v}(0) = (0, v_c) \tag{1.9}$$

とすれば、反時計回りの円軌道です。採用する条件を表1.1にまとめます。数値はこの練習のための設定であり、実在する特定の銀河を表してはいません。

表 1.1　本章の標準計算条件。最終時刻は、初期条件から求めた解析的な 1 周期です。

量	設定または計算値
中心質量 M	$10^{11}\,M_\odot$
初期半径 r_0	$10\,\mathrm{kpc}$
初期速度の大きさ v_c	$207.386\,\mathrm{km\,s^{-1}}$
周期 T	$0.296242\,\mathrm{Gyr}$
最終時刻 t_{end}	T
ステップ数 N_{step}	200
時間刻み $\Delta t = T/N_{\mathrm{step}}$	$1.4812\,\mathrm{Myr}$

よくある失敗：表示する速度と計算する速度

$\mathrm{km\,s^{-1}}$ の数値を、そのまま Gyr 単位の時間刻みに掛けないようにします。位置の更新で使う $\boldsymbol{v}\Delta t$ が kpc になるよう、内部速度は $\mathrm{kpc\,Gyr^{-1}}$ にそろえます。

1.4 Euler 法で時間を進める

計算機では、時刻を $t_0, t_1, t_2, \ldots$ と区切って状態を求めます。本章では一定の時間刻み Δt を使い、$t_n = n\Delta t$ とします。数値計算で得た状態を $\boldsymbol{y}^n$ と書きます。上付きの n は時刻の番号であり、べき乗ではありません。

時間積分の考え方を、最も簡単な Euler 法で確認しましょう。小さな時間 Δt の間では、現在の変化率を一定と近似できます。Taylor 展開の最初の 2 項だけを残すと、

$$\boldsymbol{y}(t_n + \Delta t) = \boldsymbol{y}(t_n) + \Delta t\, \boldsymbol{f}(t_n, \boldsymbol{y}(t_n)) + O(\Delta t^2) \tag{1.10}$$

です。誤差項を省略して、次の更新式を得ます。

$$\boldsymbol{y}^{n+1} = \boldsymbol{y}^n + \Delta t\, \boldsymbol{f}(t_n, \boldsymbol{y}^n) \tag{1.11}$$

位置と速度に分ければ、$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^n$、$\boldsymbol{v}^{n+1} = \boldsymbol{v}^n + \Delta t \boldsymbol{a}(\boldsymbol{r}^n)$ です。両方の更新に、更新前の同じ状態を使います。

粒子は重力によって進行方向を変えますが、Euler 法の位置更新は、その時点の接線方向へ直進させる計算です。時間刻みが有限であれば、この近似にはずれが生じます。そのずれを次の初期値として計算を続けるため、誤差が蓄積します。円軌道が数値的には円でなくなる例を、後で実際に見ます。

Euler 法は本章では導入と比較のためだけに使います。次は、位置と速度の更新を対称に組み合わせる leapfrog 法へ進みます。

1.5 leapfrog 法で位置と速度を更新する

1.5.1 位置の更新と速度の更新を分ける

重力が粒子の速度を変え、速度が粒子の位置を変えます。加速度が位置だけで決まる式(1.5)では、この二つの働きを分けて計算できます。位置を固定して加速度を与える操作を **kick**、速度を固定して位置を動かす操作を **drift** と呼びます。ここでいう kick は計算上の速度更新であり、粒子が物理的な衝撃を受けるという意味ではありません。

長さ h の区間について、それぞれの操作は

$$\begin{aligned} &\text{kick:} \quad \boldsymbol{r}' = \boldsymbol{r}, \qquad \boldsymbol{v}' = \boldsymbol{v} + h\boldsymbol{a}(\boldsymbol{r}), \\ &\text{drift:} \quad \boldsymbol{r}' = \boldsymbol{r} + h\boldsymbol{v}, \qquad \boldsymbol{v}' = \boldsymbol{v} \end{aligned} \tag{1.12}$$

です。各操作では一方の変数を固定しているため、その部分の運動は正確に計算できます。ただし、実際の運動では位置も速度も同時に変わるので、二つの操作を組み合わせた結果は近似です。Euler 法のように片側の時刻だけを使うのではなく、前後を対称に組み合わせると精度が上がります。

1.5.2 kick–drift–kick の 3 段階で更新する

本書では、半ステップの kick、1 ステップの drift、半ステップの kick という順序を使います。これが **kick–drift–kick（KDK）形式の leapfrog 法**です。[5] 時刻 t_n の位置と速度を $\boldsymbol{r}^n, \boldsymbol{v}^n$、その位置での加速度を $\boldsymbol{a}^n = \boldsymbol{a}(\boldsymbol{r}^n)$ とします。更新式を実行する順に書くと、

$$\boldsymbol{v}^{n+1/2} = \boldsymbol{v}^n + \frac{\Delta t}{2}\boldsymbol{a}^n, \tag{1.13}$$

$$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^{n+1/2}, \tag{1.14}$$

$$\boldsymbol{a}^{n+1} = \boldsymbol{a}(\boldsymbol{r}^{n+1}), \tag{1.15}$$

$$\boldsymbol{v}^{n+1} = \boldsymbol{v}^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{a}^{n+1} \tag{1.16}$$

です。最初の kick には古い位置での加速度を、最後の kick には**新しい位置での加速度**を使います。粒子を動かした後に重力の向きと強さを計算し直すことが重要です。

上付きの $n+1/2$ は、更新の途中で使う速度を表します。これは中間時刻の速度の近似ですが、中間時刻における厳密な解ではありません。第 1 の kick だけで、粒子の位置まで半ステップ進んだわけでもありません。位置と速度を交互に進める様子から、leapfrog（かえる跳び）という名前が付いています。KDK では最後の kick まで終えると、$\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1}$ が同じ整数時刻にそろいます。

図1.1を上から下へ読み、更新する量と保持する量を確認してください。最後に求めた加速度を次のステップへ渡す関係も示しています。

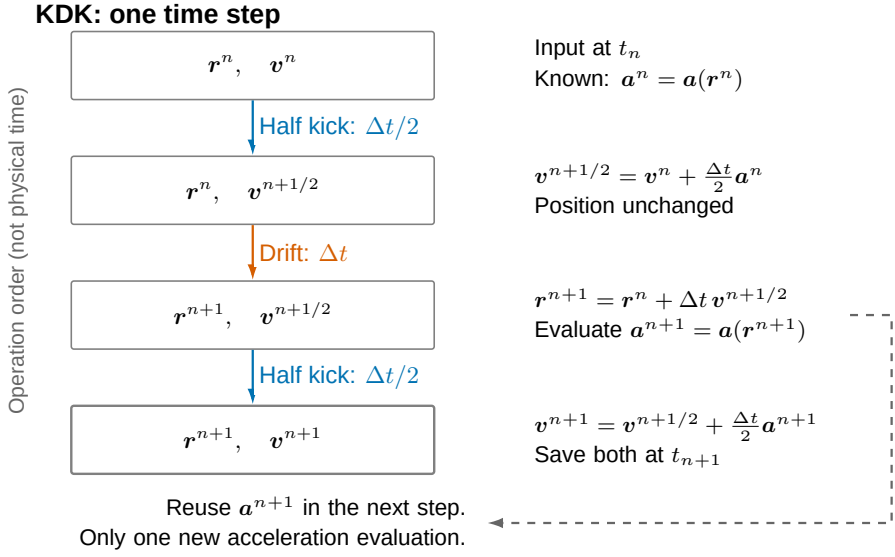


図 1.1　KDK の 1 ステップで受け渡す位置・速度・加速度。縦方向は計算操作の順序であり、物理時間軸ではありません。最後の半 kick を終えた位置と速度を整数時刻の状態として保存し、新しい加速度を次のステップへ引き継ぎます。

よくある失敗：半ステップの速度をそのまま保存しない

本書の履歴には、同じ整数時刻の位置と速度を保存します。エネルギーを計算するときに $\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1/2}$ を組み合わせると、異なる時刻の量を使ってしまいます。最後の kick を終えた $\boldsymbol{v}^{n+1}$ を用いてください。

1.5.3 2 次精度になる理由を考える

位置の更新式に最初の kick を代入すると、

$$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^n + \frac{\Delta t^2}{2}\boldsymbol{a}^n \tag{1.17}$$

となります。これは位置の Taylor 展開の 2 次までの項と一致します。Euler 法で省いていた、加速度によって軌道が曲がる効果が含まれています。

二つの kick をまとめると、速度は

$$\boldsymbol{v}^{n+1} = \boldsymbol{v}^{n} + \frac{\Delta t}{2}\left(\boldsymbol{a}^{n} + \boldsymbol{a}^{n+1}\right) \tag{1.18}$$

です。区間の両端の加速度を平均して使います。滑らかな問題では、この平均による速度更新も Taylor 展開の 2 次までと一致します。正しい初期値から 1 ステップだけ進めたときの誤差は $O(\Delta t^3)$ です。同じ有限の時刻まで計算した解の誤差は、十分小さい刻みでは $O(\Delta t^2)$ になります。この意味で leapfrog 法は **2 次精度**です。刻みを半分にすると、解の誤差はおよそ $1/4$ になると期待できます。

1.5.4 時間反転と保存系での性質を理解する

KDK は、前後が同じ形になるように操作を並べています。式(1.16)から逆順に、符号を反転した $-\Delta t$ で操作すれば、もとの状態へ戻ります。固定刻みで、丸め誤差を除けば成り立つこの性質を**時間反転対称性**と呼びます。

また、本章のような静的なポテンシャル中の運動では、KDK は**シンプレクティック積分法**です。これは Hamilton 系の位相空間の幾何学的な構造を保つ方法という意味です。本書ではその数学的な証明には立ち入りませんが、**次数だけでなく、もとの運動方程式の構造を保つことにも価値がある**と理解してください。[5]

シンプレクティックであることは、物理的なエネルギーを毎ステップ厳密に保存することとは違います。固定刻みで軌道を十分に分解できている滑らかな保存系では、エネルギー誤差が一定の範囲で振動するなど、長時間積分で有利な振る舞いが期待できます。ただし、刻みが大きすぎる計算まで正しくなるわけではなく、軌道上の位置のずれも蓄積し得ます。後で、実際の数値解を用いて確かめます。

1.5.5 加速度を再利用する

式(1.15)で求めた $\boldsymbol{a}^{n+1}$ は、次のステップの最初の kick でも使います。この値を保存して渡せば、同じ位置で加速度を計算し直す必要はありません。最初の $\boldsymbol{a}^{0}$ を 1 回求めた後は、**1 ステップにつき新しい力評価は 1 回**です。N_{step} ステップでは、全体で $N_{\mathrm{step}}+1$ 回になります。2 回の kick を行うことと、2 回の新しい力評価を行うことは同じではありません。

N 体計算では、多数の粒子が作る重力場を求める処理が高価です。加速度を再利用できることは、保存系に適した性質とともに、重力計算で leapfrog 法を使う重要な理由です。[5] Python でも、この利点が実際に得られる形で実装します。

1.6 4 次 Runge–Kutta 法で運動方程式を解く

leapfrog は、加速度が位置から決まる重力計算を効率よく進める方法です。一方、速度に依存する力を扱うときには、状態全体の変化率を使う方法が便利です。本書では、そのための主要手法として RK4 も本文で学びます。第 6 章の動力学的摩擦を含む発展計算で、この方法を使います。

1.6.1 1 ステップの途中の状態を予測する

状態の変化を正確に書けば、次の時刻の状態は

$$\boldsymbol{y}(t_n + \Delta t) = \boldsymbol{y}(t_n) + \int_{t_n}^{t_n+\Delta t} \boldsymbol{f}(t, \boldsymbol{y}(t))\, dt \tag{1.19}$$

です。しかし、積分区間の途中の $\boldsymbol{y}(t)$ は、まだ分かっていません。RK4 では途中の状態を順に予測し、そこで右辺を評価します。高階の時間微分を自分で計算する代わりに、右辺関数 $\boldsymbol{f}$ を 4 回使います。[3]

本書で使う古典的 RK4 の 4 つの段階は、

$$\begin{aligned}
\boldsymbol{k}_1 &= \boldsymbol{f}(t_n, \boldsymbol{y}^n), \\
\boldsymbol{k}_2 &= \boldsymbol{f}\left(t_n + \frac{\Delta t}{2}, \boldsymbol{y}^n + \frac{\Delta t}{2}\boldsymbol{k}_1\right), \\
\boldsymbol{k}_3 &= \boldsymbol{f}\left(t_n + \frac{\Delta t}{2}, \boldsymbol{y}^n + \frac{\Delta t}{2}\boldsymbol{k}_2\right), \\
\boldsymbol{k}_4 &= \boldsymbol{f}\left(t_n + \Delta t, \boldsymbol{y}^n + \Delta t \boldsymbol{k}_3\right)
\end{aligned} \tag{1.20}$$

です。得られた 4 つの傾きを重み付きで平均し、

$$\boxed{\boldsymbol{y}^{n+1} = \boldsymbol{y}^n + \frac{\Delta t}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right)} \tag{1.21}$$

で 1 ステップを進めます。ここでは $\boldsymbol{k}_j$ を**状態の時間変化率**として定義し、Δt を含めていません。

各段階を、物理的な計算として読み直してみましょう。$\boldsymbol{k}_1$ は現在の状態での変化率です。それを使って半ステップ後の状態を予測し、$\boldsymbol{k}_2$ を求めます。次に $\boldsymbol{k}_2$ を使い、**もう一度、現在の状態 $\boldsymbol{y}^n$ から**半ステップ後を予測して、$\boldsymbol{k}_3$ を求めます。最後に $\boldsymbol{k}_3$ を使って、現在から 1 ステップ後の状態を予測し、$\boldsymbol{k}_4$ を求めます。

$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ の評価時刻は同じですが、使う予測状態は異なります。また、途中で得る状態は、その時刻の確定した数値解ではありません。1 ステップ全体を進めるための、一時的な予測値です。

図1.2では、予測状態と、そこで計算する傾きを分けて示しています。特に第 2・第 3 段階で、時刻は同じでも状態が異なる点を見てください。

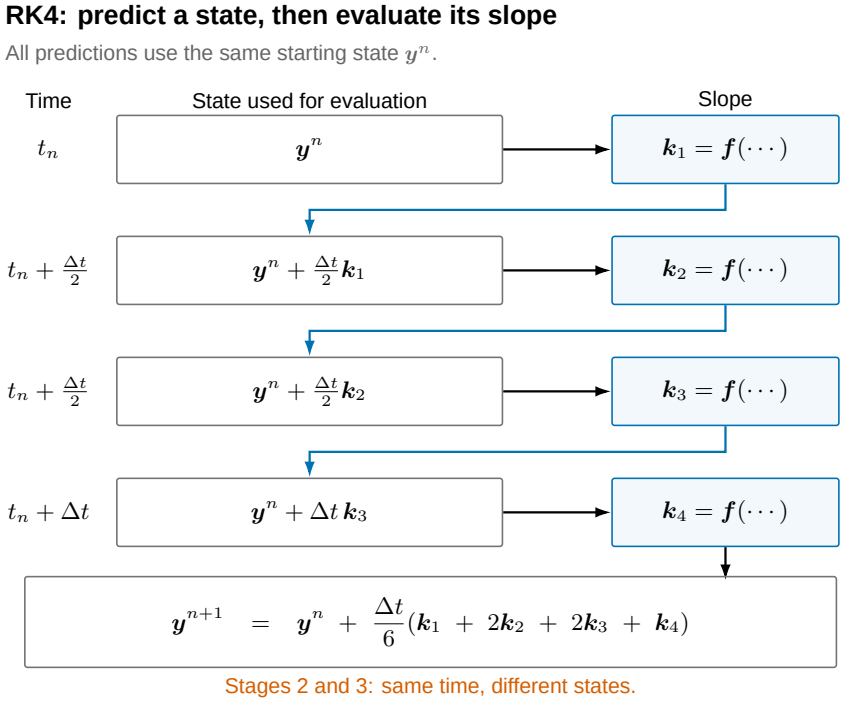


図 1.2　RK4 の予測状態と傾きの依存関係。各予測は同じ基準状態 $\boldsymbol{y}^n$ から作ります。$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ は評価時刻が同じでも、使う予測状態は異なります。矢印は計算の依存関係で、時間を 4 等分する図ではありません。

1.6.2 同じ予測状態から位置と速度の変化率を評価する

重力問題では、状態の上半分が位置、下半分が速度です。例えば $\boldsymbol{k}_1$ は $(\boldsymbol{v}^n, \boldsymbol{a}(\boldsymbol{r}^n))$ に対応します。第 2 段階で右辺を評価する位置と速度は、

$$\boldsymbol{r}_{(2)} = \boldsymbol{r}^n + \frac{\Delta t}{2}\boldsymbol{v}^n, \qquad \boldsymbol{v}_{(2)} = \boldsymbol{v}^n + \frac{\Delta t}{2}\boldsymbol{a}(\boldsymbol{r}^n) \tag{1.22}$$

です。そして、$\boldsymbol{k}_2$ の位置成分の変化率は $\boldsymbol{v}_{(2)}$、速度成分の変化率は $\boldsymbol{a}(\boldsymbol{r}_{(2)})$ です。位置だけを予測して、速度は初めのまま使う計算ではありません。

式(1.21)の係数 $1, 2, 2, 1$ は、単に「中点を重くする」という思いつきで決めたものではありません。途中の予測状態の作り方と合わせて、真の解の Taylor 展開と 4 次まで一致するように選ばれています。一般の場合の次数条件の導出は、数値計算の専門書に譲ります。[3]

1.6.3 4 次精度の意味を確認する

正確な初期値から 1 ステップだけ進めたとき、十分滑らかな問題での RK4 の誤差は $O(\Delta t^5)$ です。一方、同じ有限の時間まで多数のステップを積み重ねたときの解の誤差は、十分小さな時間刻みでは $O(\Delta t^4)$ になります。この後者を指して、4 次精度と呼びます。

したがって、初期条件と最終時刻を固定して時間刻みを半分にすると、解の誤差はおよそ 1/16 になることが期待できます。これは誤差の主要な項が Δt^4 に比例する範囲での性質です。刻みが大きすぎる場合や、丸め誤差が目立つほど小さい場合まで保証する関係ではありません。また、エネルギーのような特定の量の誤差が、必ず同じべきで変わるとも限りません。

よくある失敗：RK4 の段階と時間刻みを混同する

本書の $\boldsymbol{k}_j$ に Δt は含まれません。別の資料で $\Delta t \boldsymbol{f}$ を $\boldsymbol{k}_j$ と定義している場合、その更新式をそのまま混ぜると時間刻みを二重に掛けてしまいます。また、4 段階で位置と速度を同じ予測状態から評価することを守ります。

1.6.4 速度に依存する力も同じ形で扱う

加速度が $\boldsymbol{a}(\boldsymbol{r}, \boldsymbol{v}, t)$ となっても、RK4 では各段階の位置と速度から右辺を評価します。状態ベクトルと 4 段階の更新式を変更する必要はありません。一方、ここまでの KDK では、位置を固定すれば kick の間の加速度も固定できることを使いました。速度依存力では kick の間にも加速度が変わるので、同じ陽的な更新をそのまま使うことはできません。

これは「leapfrog の考え方では速度依存力を一切扱えない」「RK4 だけが使える」という意味ではありません。分割法や陰的な速度更新を組み合わせる方法もありますが、本書では複数の方法を追加せず、RK4 へ切り替えます。RK4 を採用する理由は、速度依存力を含む運動方程式を、学習済みの同じ枠組みで扱えることです。

もう少し詳しく：本文で詳しく扱わない時間積分法

RK2 は、少ない段階で 2 次精度を得る Runge–Kutta 法です。**適応刻み RK 法**は、推定誤差に応じて時間刻みを変えます。**Bulirsch–Stoer 法**は、複数の刻みで計算した解から外挿を行う方法です。また、kick と drift の組合せから**高次のシンプレクティック法**を作ることもできます。本書では KDK と RK4 の基本的な性質に集中するため、高次の方法の詳細は数値計算法の参考図書に譲ります。高次のシンプレクティック法については文献 [5] も参照してください。

1.7 Python で軌道を計算する

1.7.1 状態を配列へ対応させる

まず、`import numpy as np`と`import matplotlib.pyplot as plt`を実行します。本章では 1 個の粒子でも、位置と速度をそれぞれ形が`(1, 2)`の配列に入れます。最初の軸が粒子、2 番目の軸が x, y の成分です。後の章で粒子数を N へ増やしても、`(N, 2)`という同じ約束を使います。

次のコードで、単位変換、初期条件、計算時間を設定します。leapfrog では`pos`と`vel`を分けて更新します。RK4 では`np.column_stack`で位置と速度を横

に並べ、[x, y, vx, vy]を 1 行に持つ形(1, 4)のstateを使います。これはデータの持ち方の違いであり、計算する運動方程式は同じです。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass = 1.0e11                        # 中心質量 [Msun]
r0 = 10.0                            # 初期半径 [kpc]
mu = G * mass
vc = np.sqrt(mu / r0)                # 円運動速度 [kpc/Gyr]
period = 2.0 * np.pi * r0 / vc       # 軌道周期 [Gyr]
pos0 = np.array([[r0, 0.0]])         # shape = (1, 2)
vel0 = np.array([[0.0, vc]])         # shape = (1, 2)
state0 = np.column_stack((pos0, vel0))  # RK4 用：[x, y, vx, vy]
n_steps = 200
t_end = period
dt = t_end / n_steps
```

np.array([[r0, 0.0]])の角括弧が二重なのは、粒子の行と成分の列を作るためです。pos0[0]で最初の粒子の位置ベクトルを取り出せます。式(1.4)を、まず 1 粒子の成分のまま計算します。

```
x, y = pos0[0]
r2 = x**2 + y**2
ax = -mu * x / r2**1.5
ay = -mu * y / r2**1.5
print(f"ax = {ax:.6f}, ay = {ay:.6f} [kpc/Gyr^2]")
```

初期位置では、出力される a_x は $-4.4985 \times 10^3\,\mathrm{kpc\,Gyr^{-2}}$、$a_y$ は 0 です。原点へ向かう加速度になっていることを確認してください。

1.7.2 加速度関数と右辺関数を分ける

今計算した式を、位置から加速度を返す関数にします。入力と返り値は、どちらも形が(1, 2)です。この関数は本章の 1 粒子専用で、多数粒子への一般化は第 4 章で行います。

```
def acceleration_kepler(pos, mu):
    """1 粒子の重力加速度。本文 eq:ch1-acceleration-components。"""
    x, y = pos[0]
    r2 = x**2 + y**2
    if r2 <= 0.0:
        raise ValueError("Point-mass gravity is undefined at r = 0.")
    ax = -mu * x / r2**1.5
    ay = -mu * y / r2**1.5
    return np.array([[ax, ay]])
```

leapfrog には、この加速度関数を渡します。Euler 法と RK4 には、位置と速度をまとめた状態の変化率を返す関数を渡します。同じ加速度関数を使い、式(1.6)の右辺を組み立てます。

```
def rhs_kepler(t, state, mu):
    """ 状態 [x, y, vx, vy] の右辺。本文 eq:ch1-state-ode。"""
    pos, vel = state[:, :2], state[:, 2:]
    acc = acceleration_kepler(pos, mu)
    return np.column_stack((vel, acc))
```

`state[:, :2]`が位置、`state[:, 2:]`が速度です。右辺の先頭 2 成分は速度、後半 2 成分は加速度です。`t`は本章では使いませんが、時間依存する問題にも使えるように残します。

1.7.3 KDK の 1 ステップを実装する

式(1.13)から式(1.16)までを、その順に書きます。引数`acc`は、現在の`pos`に対応する計算済みの加速度です。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mu):
    """KDK で 1 ステップ。acc は現在位置での加速度です。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mu)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

`vel_half`は中間速度、`pos_new`と`vel_new`は次の整数時刻の状態です。返り値の`acc_new`を次の呼び出しへ渡します。関数内部の`acceleration(...)`は 1 回だけです。入力配列を直接上書きせず、新しい配列を返すため、比較計算でも初期値を保てます。

確認：最後の kick で`acc_new`ではなく古い`acc`を使うと、新しい位置で変化した重力を反映できません。また、`acc`と`pos`の対応がずれると、意図した KDK ではなくなります。

1.7.4 Euler 法と RK4 の 1 ステップを実装する

Euler 法は、状態全体に更新前の変化率を掛けるだけです。

```
def euler_step(rhs, t, state, dt, mu):
    return state + dt * rhs(t, state, mu)
```

RK4 は、式(1.20)と式(1.21)を順に実装します。

```
def rk4_step(rhs, t, state, dt, mu):
    """RK4 の 4 段階。本文 eq:ch1-rk4-stages, eq:ch1-rk4-update。"""
    k1 = rhs(t, state, mu)
    k2 = rhs(t + 0.5 * dt, state + 0.5 * dt * k1, mu)
    k3 = rhs(t + 0.5 * dt, state + 0.5 * dt * k2, mu)
    k4 = rhs(t + dt, state + dt * k3, mu)
    return state + dt * (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
```

`state + 0.5 * dt * k1`には、予測位置と予測速度が一緒に入ります。RK4 の段階全体で同じ予測状態を使う点は、位置と速度を分けて更新する KDK との違いです。

1.7.5 更新を繰り返して履歴を残す

KDK では、最初の加速度をループの外で 1 回だけ計算します。その後は、位置・速度・加速度を受け取り、次のステップへ渡します。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mu):
    """ 初回の力を計算し、以降は前ステップの acc_new を使います。"""
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mu)  # 初回だけ、ループの前に計算します。
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mu)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the time step.")
    return times, history
```

`prepare_history`は、正のステップ数、有限の最終時刻、初期配列の形を確認する補助関数です。初期状態のコピー、$\Delta t = t_{\mathrm{end}}/N_{\mathrm{step}}$、$t_n = n\Delta t$、保存用配列を用意します。その実装は Notebook に示します。`history`の形は`(n_steps + 1, 1, 4)`です。初期状態を含むので、出力数はステップ数より 1 つ多くなります。KDK の履歴にも、**位置と最後の kick 後の速度**を同じ行へ保存します。

Euler 法と RK4 では、一般の右辺と 1 ステップ関数を渡して計算します。

```
def integrate(rhs, stepper, state0, t_end, n_steps, mu):
    """Euler 法・RK4 用。右辺関数を指定して等間隔で積分します。"""
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    for n in range(n_steps):
        state = stepper(rhs, times[n], state, dt, mu)
        if not np.all(np.isfinite(state)):
            raise FloatingPointError("Non-finite state; check the time step.")
        history[n + 1] = state
    return times, history
```

同じ初期条件から、3 つの方法を実行します。本章では変数`history`に標準法の leapfrog の結果を入れ、RK4 と Euler 法には別の変数を使います。

```
times, history = integrate_leapfrog(acceleration_kepler, state0, t_end, n_steps, mu)
_, history_euler = integrate(rhs_kepler, euler_step, state0, t_end, n_steps, mu)
_, history_rk4 = integrate(rhs_kepler, rk4_step, state0, t_end, n_steps, mu)
```

1.8 軌道を図とアニメーションで見る

円軌道を初期条件に選んだので、時刻 t における正解は、初期条件から独立に計算できます。角速度 Ω を使うと、位置と速度は

$$\begin{aligned} \boldsymbol{r}_{\mathrm{exact}}(t) &= r_0\bigl(\cos\Omega t,\, \sin\Omega t\bigr), \\ \boldsymbol{v}_{\mathrm{exact}}(t) &= r_0\Omega\bigl(-\sin\Omega t,\, \cos\Omega t\bigr) \end{aligned} \tag{1.23}$$

です。数値解と解析解を、必ず**同じ時刻**で比べます。単にどちらも円を描いているだけでは、1 周する速さが正しいかは分かりません。

解析解を計算するコードは次のようになります。ここでpos_historyの最初の軸は、粒子数ではなく、保存した時刻を表しています。

```
def exact_circle(times, r0, mu):
    omega = np.sqrt(mu / r0**3)
    phase = omega * np.asarray(times)
    pos_exact = np.column_stack((r0 * np.cos(phase), r0 * np.sin(phase)))
    vel_exact = np.column_stack((-r0 * omega * np.sin(phase), r0 * omega *
    ↪  np.cos(phase)))
    return pos_exact, vel_exact

pos_exact, vel_exact = exact_circle(times, r0, mu)
pos_history = history[:, 0, :2]  # 第 1 軸は出力時刻です。
vel_history = history[:, 0, 2:]
```

図1.3は、標準の 200 ステップで計算した leapfrog の軌道です。$0, T/4, T/2, 3T/4$ の位置も示しました。解析解と数値解は、この図の大きさではほぼ重なります。図を描くときはax.set_aspect("equal")を使い、縦横で同じ距離が同じ長さに見えるようにします。そうしないと、円を楕円のように表示してしまいます。

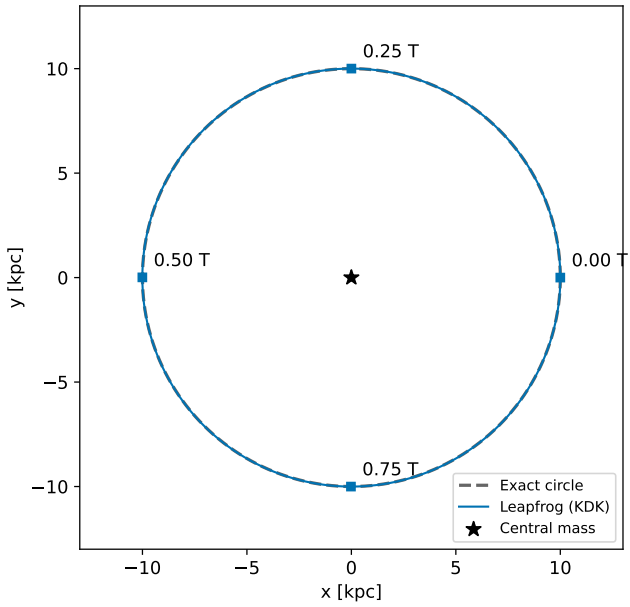


図 1.3　中心質量の周りを回る 1 粒子の軌道。標準条件の leapfrog（KDK）を青実線、解析的な円を灰色破線で示します。四角は指定した時刻の数値解、星印は固定した中心質量です。計算は反時計回りです。

描画の中心部分を次に示します。save_figureは Notebook に用意した保存用関数で、同じ図を PDF と PNG に保存して表示します。

```
fig, ax = plt.subplots(figsize=(5.6, 5.2), layout="constrained")
ax.plot(pos_exact[:, 0], pos_exact[:, 1], color=GRAY, ls="--", lw=2, label="Exact
↪  circle")
ax.plot(pos_history[:, 0], pos_history[:, 1], color=BLUE, lw=1.3, label="Leapfrog (KDK)")
ax.scatter([0.0], [0.0], marker="*", s=90, color="black", label="Central mass")
sample_ids = np.rint(np.arange(4) * n_steps / 4).astype(int)
for n in sample_ids:
    ax.plot(*pos_history[n], marker="s", color=BLUE, ms=5)
    label = f"{times[n] / period:.2f} T"
    ax.annotate(label, pos_history[n], xytext=(7, 7), textcoords="offset points")
ax.set(xlabel="x [kpc]", ylabel="y [kpc]", xlim=(-13, 13), ylim=(-13, 13))
ax.set_aspect("equal")
ax.legend(loc="lower right", fontsize=9)
save_figure(fig, "ch01_orbit")
```

同じ条件で Euler 法を使った結果を、図1.4に重ねました。本来は 1 周期で初期位置へ戻りますが、Euler 法では軌道が外側へずれ、1 周期後の位置も大きく異なります。これは別の初期速度を与えた結果ではなく、**同じ運動方程式と同じ初期条件を、異なる近似で時間積分した結果**です。

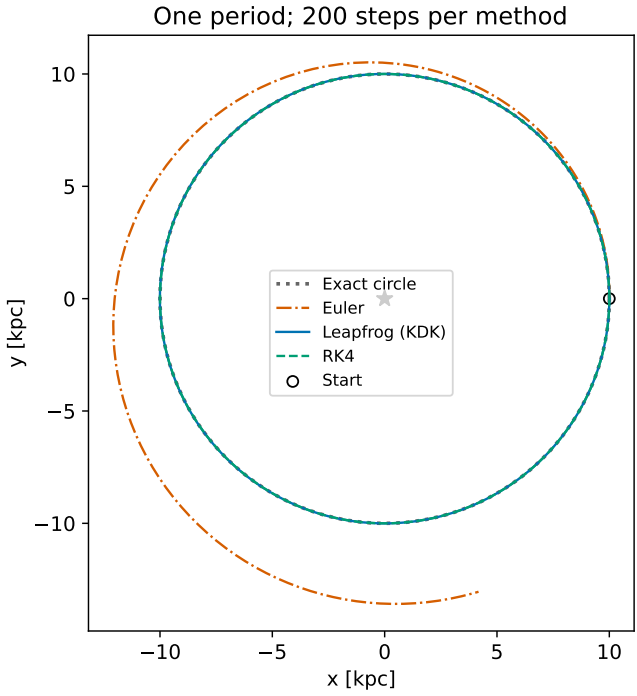


図 1.4　同じ時間刻み $\Delta t = T/200$ での Euler 法・leapfrog・RK4 の比較。計算時間は解析的な 1 周期です。朱色の一点鎖線が Euler 法、青実線が leapfrog、緑破線が RK4、灰色点線が解析解です。同じステップ数でも力評価回数は異なり、leapfrog と RK4 の軌道はこの縮尺ではほぼ重なります。

アニメーションと静止画　Notebook では、保存済みの`history`から粒子の位置を順に読み、アニメーションを作ります。表示のたびに運動方程式を解き直す必要はありません。再生間隔を決める`interval`は画面上の時間であり、物理的な時間刻み`dt`ではありません。再生速度を変えても計算した軌道は変わりません。本文では図1.3のように重要な時刻の位置を静止画としても残し、動画を見なくても結果が分かるようにします。

1.9 エネルギーと角運動量を確かめる

1.9.1 このモデルで保存される量を確認する

軌道の見た目だけでは、小さな誤差は分かりません。そこで、ゼミで学んだ保存量を数値解から計算します。中心質量は固定され、ポテンシャルは時間に依存しません。粒子の単位質量当たりのエネルギーは

$$\varepsilon = \frac{1}{2}(v_x^2 + v_y^2) - \frac{\mu}{r} \tag{1.24}$$

です。速度変化による運動エネルギーの変化率と、位置変化によるポテンシャルエネルギーの変化率が打ち消し合い、

$$\frac{d\varepsilon}{dt} = \boldsymbol{v}\cdot\boldsymbol{a} + \frac{\mu}{r^3}\boldsymbol{r}\cdot\boldsymbol{v} = 0 \tag{1.25}$$

となります。

また、重力は原点からの位置ベクトルと平行で、原点の周りのトルクは 0 です。運動面に垂直な単位質量当たりの角運動量は

$$\ell_z = x v_y - y v_x, \qquad \frac{d\ell_z}{dt} = x a_y - y a_x = 0 \tag{1.26}$$

となります。ここで確認するのは粒子の**比**エネルギーと**比**角運動量であり、粒子の質量 m は掛けません。これらは、中心力中の軌道を理解する基本的な保存量です。[1]

円軌道の初期値では、

$$\varepsilon_0 = -\frac{\mu}{2r_0}, \qquad \ell_{z,0} = r_0 v_c = \sqrt{\mu r_0} \tag{1.27}$$

です。負のエネルギーは、この粒子が中心質量に束縛されていることを表します。今回は ε_0 も $\ell_{z,0}$ も 0 ではないので、それぞれの初期値の絶対値で誤差を規格化できます。

1.9.2 数値解から保存量を計算する

履歴から位置と速度を取り出し、式(1.24)と式(1.26)を評価します。`np.sum(..., axis=1)` は、ここでは各時刻の 2 成分を足す操作です。例えば `np.sum(vel**2, axis=1)` は、各時刻の $v_x^2 + v_y^2$ を返します。

```python
def diagnostics(history, mu):
    pos = history[:, 0, :2]
    vel = history[:, 0, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=1))
    energy = 0.5 * np.sum(vel**2, axis=1) - mu / radius
    angular_momentum = pos[:, 0] * vel[:, 1] - pos[:, 1] * vel[:, 0]
    return radius, energy, angular_momentum
```

時刻 t_n での誤差の大きさを、

$$e_\varepsilon^n = \frac{|\varepsilon^n - \varepsilon^0|}{|\varepsilon^0|}, \qquad e_\ell^n = \frac{|\ell_z^n - \ell_z^0|}{|\ell_z^0|} \tag{1.28}$$

と定義します。保存した全時刻での最大値を、それぞれ e_ε, e_ℓ とします。図1.5は、エネルギー誤差を時間の関数として示したものです。$t = 0$ では定義上誤差が 0 です。対数軸では 0 を表示できないため、その点だけを描いていません。保存した数値を、小さな正数へ置き換えているわけではありません。

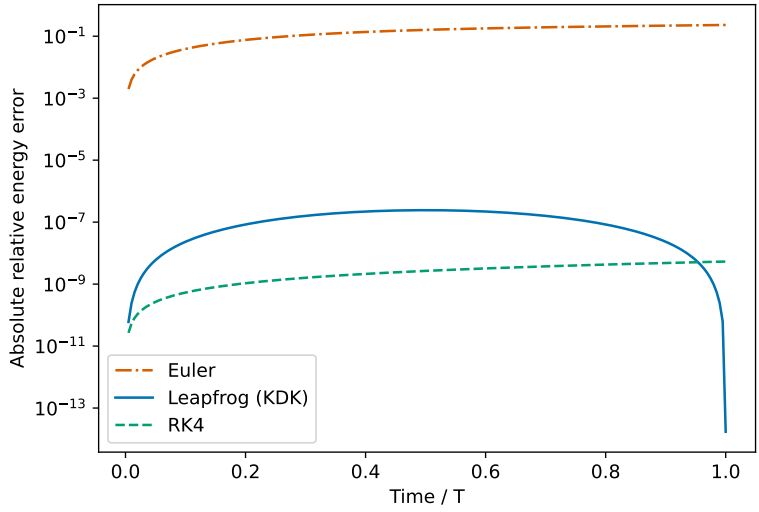


図 1.5　標準条件でのエネルギーの相対誤差。各方法の初期値からの変化の絶対値を示しています。縦軸は対数です。Euler 法、leapfrog、RK4 を比べています。leapfrog も RK4 もエネルギーを厳密に保存するわけではありません。

標準条件の誤差を表1.2にまとめました。位置の誤差 e_r は、解析解との位置の差を初期半径で割った最大値です。次節で詳しく定義します。

leapfrog のエネルギー誤差は約 2.43×10^{-7} です。一方、角運動量のずれは約 6.43×10^{-16} で、丸め誤差程度にとどまります。これは、中心力による kick と drift の各操作が、角運動量を保つためです。例えば kick では $\boldsymbol{r} \times \boldsymbol{a} = 0$ なので、速度に $h\boldsymbol{a}$ を加えても $\boldsymbol{r} \times \boldsymbol{v}$ は変わりません。drift でも、位置へ $h\boldsymbol{v}$ を加えた寄与は $h\boldsymbol{v} \times \boldsymbol{v} = 0$ です。この説明は本章の中心力についてのもので、任意の外力に角運動量保存を保証するものではありません。

表 1.2　同じ 1 周期・200 ステップでの比較。e_r は最大位置誤差、e_ε と e_ℓ は保存量の最大相対誤差です。本章の実行結果であり、末尾の桁は実行環境で多少変わります。

方法	力評価回数	e_r	e_ε	e_ℓ
Euler	200	1.43×10^{0}	2.31×10^{-1}	1.38×10^{-1}
Leapfrog (KDK)	201	2.16×10^{-3}	2.43×10^{-7}	6.43×10^{-16}
RK4	800	1.65×10^{-7}	5.34×10^{-9}	2.67×10^{-9}

RK4 の最大相対エネルギー誤差は 5.34×10^{-9}、角運動量誤差は 2.67×10^{-9} でした。短時間の位置とエネルギーの精度は非常に高い一方、エネルギーや角運動量を厳密に保存する手法ではありません。[3] また、どの方法でも、保存量のずれが小さいことだけで軌道上の位置が正しいとは判断できません。次節では、解析解に対する位置ベクトルの誤差を直接調べます。

よくある失敗：保存量はモデルを見て選ぶ

本章では、静的な中心重力場なのでエネルギーと角運動量を確認できます。後の章のように、重力場が時間変化する場合や摩擦がある場合には、粒子のエネルギーは一般に保存しません。「エネルギーが変わったからコードが間違っている」と決める前に、計算したモデルで何が保存されるかを確認します。

1.10 時間刻みを変えて誤差を調べる

1.10.1 位置ベクトルの誤差を測る

時間刻みの影響を分けて調べるため、中心質量、初期位置、初期速度、最終時刻 T を固定します。変えるのはステップ数だけです。ステップ数を 2 倍にすると、時間刻みは半分になります。数値解と解析解を同じ時刻で比較し、位置の差を r_0 で規格化します。保存した全時刻での最大値を

$$e_r = \max_{0 \le n \le N_{\mathrm{step}}} \frac{|\boldsymbol{r}^n - \boldsymbol{r}_{\mathrm{exact}}(t_n)|}{r_0} \tag{1.29}$$

と定義します。これは半径の差だけではなく、位置**ベクトル**の差です。円に沿っていても回転の速さがずれていれば、誤差に現れます。同様に、式(1.28)の最大値を e_ε, e_ℓ とします。

```
def error_metrics(times, history, mu, r0):
    pos_exact, _ = exact_circle(times, r0, mu)
    radius, energy, angular_momentum = diagnostics(history, mu)
    position_error = np.linalg.norm(history[:, 0, :2] - pos_exact, axis=1) / r0
    energy_error = np.abs(energy - energy[0]) / abs(energy[0])
    angular_error = np.abs(angular_momentum - angular_momentum[0]) /
    ↪  abs(angular_momentum[0])
    return np.max(position_error), np.max(energy_error), np.max(angular_error)
```

同じステップ数で leapfrog と RK4 を計算し、その結果を表にまとめます。

```
step_counts = [20, 40, 80, 160, 320]
rows_lf, rows_rk4 = [], []
for count in step_counts:
    t, h_lf = integrate_leapfrog(acceleration_kepler, state0, period, count, mu)
    _, h_rk4 = integrate(rhs_kepler, rk4_step, state0, period, count, mu)
    rows_lf.append([count, period / count, *error_metrics(t, h_lf, mu, r0)])
    rows_rk4.append([count, period / count, *error_metrics(t, h_rk4, mu, r0)])
convergence_lf, convergence_rk4 = np.array(rows_lf), np.array(rows_rk4)
orders_lf = np.log2(convergence_lf[:-1, 2] / convergence_lf[1:, 2])
orders_rk4 = np.log2(convergence_rk4[:-1, 2] / convergence_rk4[1:, 2])
```

表 1.3　時間刻みと最大位置誤差。すべて同じ初期条件から 1 周期 T まで計算しています。p_r は一つ前の行との誤差比から求めた見かけの次数です。保存量の誤差も、配布する CSV に保存しています。

N_{step}	Δt [Myr]	e_r (KDK)	p_r	e_r (RK4)	p_r
20	14.8121	2.09×10^{-1}	—	3.91×10^{-3}	—
40	7.4060	5.35×10^{-2}	1.967	1.66×10^{-4}	4.557
80	3.7030	1.35×10^{-2}	1.990	7.93×10^{-6}	4.387
160	1.8515	3.37×10^{-3}	1.998	4.19×10^{-7}	4.242
320	0.9258	8.44×10^{-4}	1.999	2.40×10^{-8}	4.129

1.10.2 2 次精度と 4 次精度を確認する

刻みを半分にした二つの計算から、見かけの次数を

$$p_r = \frac{\log\left[e_r(\Delta t)/e_r(\Delta t/2)\right]}{\log 2} \tag{1.30}$$

で求めます。leapfrog では 2 へ、RK4 では 4 へ近づくと期待できます。表1.3の最も細かい側では、それぞれ 1.999 と 4.129 です。図1.6では、同じ数値を両対数の図にしています。点線と一点鎖線は、それぞれ傾き 2 と 4 の参照線です。

粗い刻みでは高次の誤差項も影響するため、見かけの次数は 2 や 4 から離れます。一方、極端に細かい刻みでは、浮動小数点演算の丸め誤差が目立つことがあります。期待した誤差比は、刻みが十分小さく、かつ丸め誤差が支配しない範囲で確認します。

円軌道では、エネルギーなど特定の量の誤差に打ち消しが生じる場合があります。その量だけから「leapfrog は 4 次」「RK4 は 5 次」と判断してはいけません。本章では、同じ時刻の解析解に対する位置のずれを、解法の次数を確認する指標にしました。

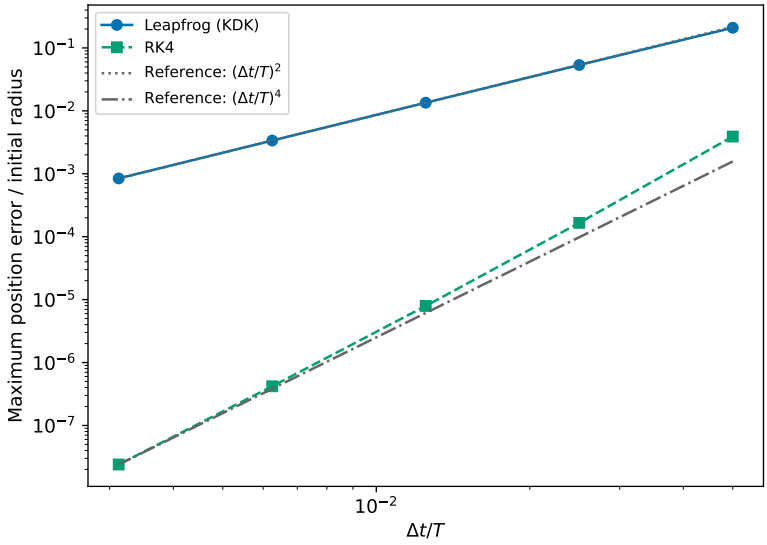


図 1.6　時間刻みと最大位置誤差の関係。青丸が leapfrog、緑四角が RK4 です。参照線は各方法の最も細かい刻みの実測点を通り、傾きを 2 または 4 に固定した線です。データに当てはめて求めた線ではありません。

1.11 計算コストと長時間の振る舞いを比較する

1.11.1 力評価回数をそろえる

同じ時間刻みで比べれば、RK4 の方が小さい位置誤差を与えました。ただし、RK4 は 1 ステップで右辺を 4 回評価し、各評価で重力を計算します。キャッシュ付き KDK は、最初の 1 回を除けば新しい重力評価が 1 回だけです。N_{step} ステップの力評価回数は

$$N_{\mathrm{force}} = \begin{cases} N_{\mathrm{step}} & \text{Euler}, \\ N_{\mathrm{step}} + 1 & \text{leapfrog（KDK）}, \\ 4N_{\mathrm{step}} & \text{RK4} \end{cases} \tag{1.31}$$

です。ここでは時間積分のための力評価だけを数え、保存量の計算や描画は含めません。200 ステップの場合の 200 回、201 回、800 回という値は、配布コードの独立試験でも関数の呼び出し数から確認しています。

力評価回数を 800 回にそろえる比較も行います。KDK は初回の 1 回が必要なので 799 ステップ、RK4 は 200 ステップです。いずれも同じ 1 周期 T まで進めます。

```
force_budget = 800
n_lf_cost = force_budget - 1
n_rk_cost = force_budget // 4
t_lf_cost, h_lf_cost = integrate_leapfrog(acceleration_kepler, state0, period, n_lf_cost, mu)
t_rk_cost, h_rk_cost = integrate(rhs_kepler, rk4_step, state0, period, n_rk_cost, mu)
cost_lf_errors = error_metrics(t_lf_cost, h_lf_cost, mu, r0)
cost_rk4_errors = error_metrics(t_rk_cost, h_rk_cost, mu, r0)
```

表 1.4 力評価回数を 800 回にそろえた 1 周期の比較。時間刻みは同じではありません。e_r は最大位置誤差、e_ε は最大相対エネルギー誤差です。

方法	N_{step}	力評価回数	e_r	e_ε
Leapfrog (KDK)	799	800	1.35×10^{-4}	9.56×10^{-10}
RK4	200	800	1.65×10^{-7}	5.34×10^{-9}

この滑らかな円軌道の短時間計算では、力評価回数をそろえても RK4 の位置誤差の方が小さくなりました。したがって、**力評価が少ないから、常に leapfrog の方が高精度になるわけではありません**。必要な精度、計算する時間の長さ、保存系としての性質を合わせて考えます。

直接 N 体計算では、1 回の重力評価に多数の粒子対の計算が含まれます。粒子数を N とすると、直接和の計算量は $O(N^2)$ です。本書のテスト粒子モデルは少数のポテンシャルから力を受けるので $O(N)$ ですが、どちらでも力評価の節約には意味があります。ただし、実行時間には配列の作成や Python の処理も含まれます。力評価回数だけから「実行時間が必ず 4 倍違う」とは言えません。Notebook には実行時間を測る例も用意し、環境ごとに確かめます。

1.11.2 同じ円軌道を 100 周期まで進める

次に、円軌道を長く追跡します。初期条件は同じですが、この実験だけ時間刻みを $\Delta t = T/100$、最終時刻を $100T$ とします。短時間の表と混同しないよう、条件の違いを明示しておきます。両方法とも 10000 ステップで、力評価回数は KDK が 10001 回、RK4 が 40000 回です。

エネルギーが一方向へずれるのか、増減を繰り返すのかを見るため、ここでは絶対値ではなく、**符号付き**の差 $[\varepsilon(t) - \varepsilon(0)]/|\varepsilon(0)|$ を描きます。

```
n_periods = 100
steps_per_period = 100
n_long = n_periods * steps_per_period
t_long, h_long_lf = integrate_leapfrog(acceleration_kepler, state0, n_periods * period, n_long, mu)
_, h_long_rk4 = integrate(rhs_kepler, rk4_step, state0, n_periods * period, n_long, mu)
_, e_long_lf, l_long_lf = diagnostics(h_long_lf, mu)
_, e_long_rk4, l_long_rk4 = diagnostics(h_long_rk4, mu)
signed_lf = (e_long_lf - e_long_lf[0]) / abs(e_long_lf[0])
signed_rk4 = (e_long_rk4 - e_long_rk4[0]) / abs(e_long_rk4[0])
```

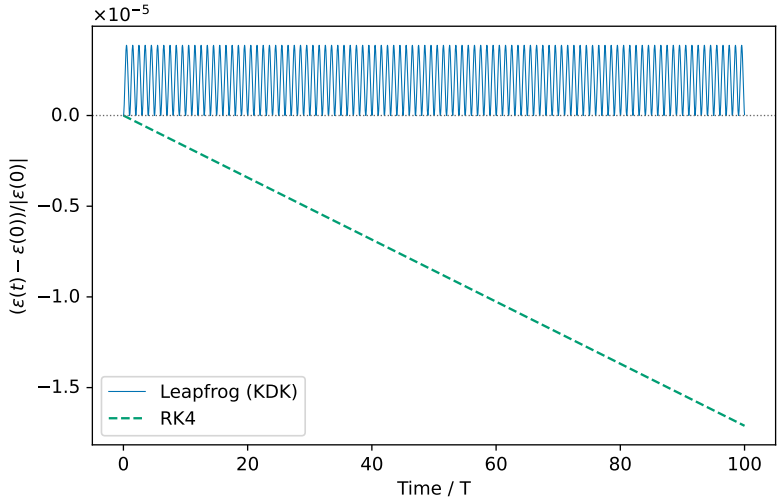


図 1.7 円軌道の 100 周期積分。両方法とも $\Delta t = T/100$ です。青実線が KDK、緑破線が RK4 です。KDK の細かい振動は、この縮尺では帯のように見えます。同じ時間刻みでの比較であり、同じ力評価回数での比較ではありません。

この条件では、KDK のエネルギー差は 0 付近から約 3.87×10^{-6} までの範囲で振動しました。RK4 の差は負の向きへ蓄積し、$100T$ で約 -1.71×10^{-5} となりました。これは本章の実行結果であり、任意の軌道や刻みで同じ数値や符号になるという主張ではありません。

静的な滑らかな保存系を固定刻みで解くとき、leapfrog の構造を保つ性質は、このような長時間の誤差の振る舞いに利点を持ちます。[5] ただし、エネルギー差が狭い範囲に収まっていても、軌道の位相が正確とは限りません。精度の確認には、保存量と位置の誤差を併用します。

1.11.3 本書での使い分けを考える

第 2 章から第 5 章までの重力計算では、leapfrog（KDK）を標準とします。計算コストを抑えながら重力系を追跡する方法として、実装を繰り返し使います。RK4 は、一般の常微分方程式を解くもう一つの主要手法として理解し、第 6 章の速度依存力を含む発展問題で用います。

そこでは Chandrasekhar の動力学的摩擦を、質量をもつ衛星銀河・銀河中心の運動へ加えます。質量ゼロの恒星テスト粒子そのものに摩擦を掛けるわけではありません。本章末の速度比例抵抗は、その前に速度依存力を扱うための小さな練習です。**手法は次数の高さだけでなく、方程式の形、必要な精度、計算コストに応じて選びます。**

1.12 本章のまとめ

本章では、固定した中心質量の周りを動く 1 個の粒子を計算しました。位置と速度を変数として持つと、Newton の運動方程式を 1 階の連立常微分方程式として扱えます。加速度を計算する部分と時間を進める部分を分けることで、同じ問題に異なる積分法を適用できました。

Euler 法で時間を刻む考え方を確認した後、重力計算の標準法として leapfrog（KDK）を実装しました。半 kick、drift、半 kick という順序、整数時刻にそろえた速度の保存、加速度の再利用が重要です。一般の常微分方程式へ進むため、RK4 の 4 段階も学びました。RK4 は、速度依存力を含む発展問題で再利用します。

両者を、軌道の見た目、解析解との位置の差、エネルギー、角運動量、時間刻み、力評価回数から比較しました。leapfrog は 2 次、RK4 は 4 次ですが、次数だけで優劣は決まりません。長時間の保存特性と、必要な精度を得るための計算コストも判断材料です。**計算が最後まで動いたことと、目的に必要な精度が得られたことは同じではありません**。次の章では leapfrog を使い、2 つの天体が互いの重力で動く問題へ進みます。

演習

演習 1.1　確認：初期値と単位　表1.1の条件から、$v_c, T, \varepsilon_0, \ell_{z,0}$ を計算してください。v_c は $\mathrm{km\,s^{-1}}$ と $\mathrm{kpc\,Gyr^{-1}}$ の両方で示し、保存量にも単位を付けてください。

演習 1.2　確認：KDK の更新と力評価　KDK の 3 段階を式で書き、各 kick で使う加速度がどの位置に対応するか説明してください。200 ステップの力評価回数を求めてください。また、$\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1/2}$ からエネルギーを計算する場合の問題点を説明してください。

演習 1.3　確認：RK4 の中間状態　$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ の評価時刻と予測状態を、それぞれ式で書いてください。$\boldsymbol{k}_2$ を求めるとき、位置成分の時間変化率に $\boldsymbol{v}^n$ をそのまま使うと、何が間違うでしょうか。

演習 1.4　応用：ステップ数の変更　初期条件と最終時刻 T を固定し、$N_{\mathrm{step}} = 50, 100, 200$ で leapfrog と RK4 を実行してください。最大位置誤差 e_r を求め、ステップ数を 2 倍にした誤差比を調べてください。軌道の図だけで精度を区別できるかも考えてください。

演習 1.5　応用：回転方向の変更　初期条件の v_y だけを $-v_c$ へ変え、leapfrog で同じ時間まで積分してください。軌道、エネルギー、角運動量はどのように変わるでしょうか。数値解が x 軸について鏡映した関係になるかを確認してください。

演習 1.6　応用：同じ計算コストでの比較　時間積分の力評価を両方とも 800 回にそろえ、1 周期を計算してください。両方法のステップ数、時間刻み、位置誤差を

比べてください。同じ刻みでの比較とは、何をそろえ、何を変えている点が違いますか。

演習 1.7　発展：速度に依存する力　重力加速度に速度比例抵抗 $-\gamma \boldsymbol{v}$ を追加し、$\gamma = 0.05/T$ として、400 ステップで 1 周期分の時間まで積分してください。右辺関数だけを変更し、RK4 の関数はそのまま使います。このモデルでは

$$\frac{d\varepsilon}{dt} = -\gamma v^2 \tag{1.32}$$

となります。エネルギー変化の向きは、この式と整合するでしょうか。この抵抗力は速度依存力の練習用であり、Chandrasekhar の動力学的摩擦公式ではありません。